day02_数据同步服务

今日内容：

- 1- 【了解】数据同步服务的基本概念
- 2- 【参考文档安装成功】安装数据同步服务
- 3- 【会操作】数据同步服务RSYNC命令基本使用
- 4- 【能够自己实现案例】实时定时与增量备份
- 5- 【能够自己实现案例】基于INOTIFY实现实时同时

1. 数据同步服务基本介绍

在CentOS Stream 9中，数据同步服务Rsync是一个开源的多功能的用于在本地和远程系统之间高效同步和备份文件的工具，通过只传输变化的数据来节省带宽和时间。

主要特点：

- 支持增量同步：只传输发生变化的数据部分。
- 支持 SSH 加密传输。
- 可保持文件权限、时间戳、连接和等元数据。

核心原理：

```
    Rsync的基本原理是通过使用“差异算法”来仅同步源文件和目标文件之间的差异部分，而不是重新传输整个文件

这一过程的核心是：
    1：分块比较：Rsync将文件分为多个小块（通常是固定大小的块），然后对比源文件和目标文件中每个块的内容
    2：校验和计算：对于每个块，Rsync计算出一个校验和（checksum），然后将源文件和目标文件的校验和进行比较，判断哪些块发生了变化
    3：传输差异部分：如果某块在源文件和目标文件中不同，则只传输这一部分的差异数据，而不是整个文件
    4：优化：Rsync还支持压缩（通过-z选项）和增量传输，使得同步过程更加高效，尤其在带宽有限的情况下

    通过这种机制，Rsync大大减少了传输的数据量，从而提高了文件同步的效率，特别是在大文件和大规模目录结构中
```

数据同步任务背景：

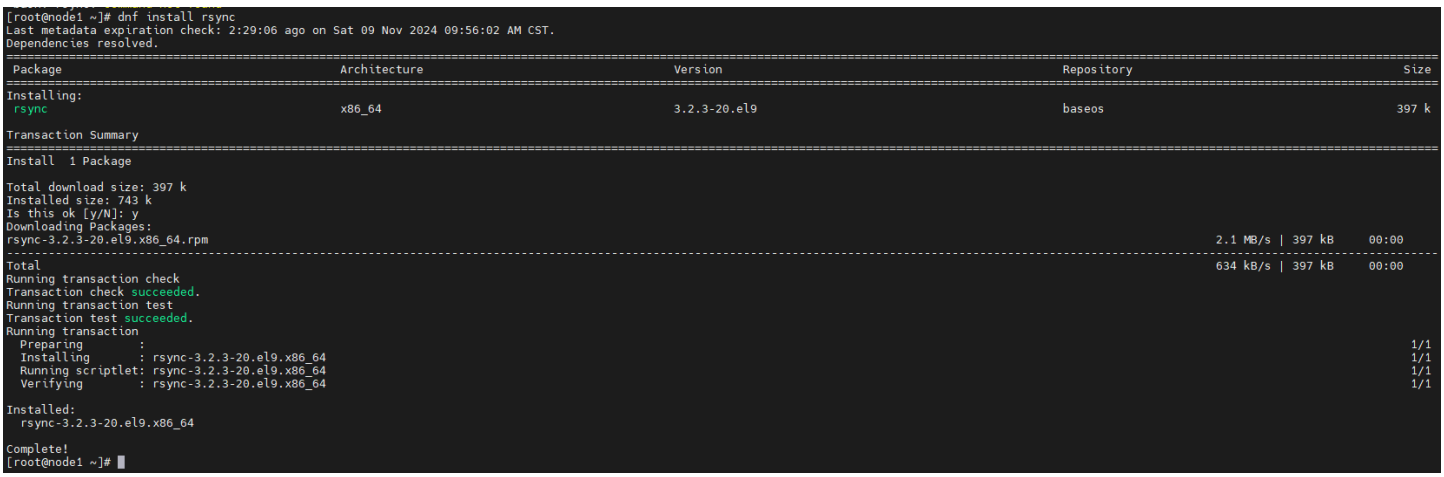
- 数据同步的重要性：
  - 确保多台服务器之间数据一致性。
  - 实现快速备份与恢复。
  - 提升运维效率，减少人为干预。
- 常见应用场景：
  - 网站内容同步。
  - 配置文件备份。
  - 数据迁移与灾备。

同步方式: 定时同步 和 实时同步

2. 安装数据同步服务

安装：

```
1   dnf install rsync -y
```



rsync --version

```
[root@node1 ~]# rsync --version
rsync  version 3.2.3  protocol version 31
Copyright (C) 1996-2020 by Andrew Tridgell, Wayne Davison, and others.
Web site: https://rsync.samba.org/
Capabilities:
    64-bit files, 64-bit inums, 64-bit timestamps, 64-bit long ints,
    socketpairs, hardlinks, hardlink-specials, symlinks, IPv6, atimes,
    batchfiles, inplace, append, ACLs, xattrs, optional protect-args, iconv,
    symtimes, prealloc, stop-at, no crtimes
Optimizations:
    SIMD, asm, openssl-crypto
Checksum list:
    md5 md4 none
Compress list:
    zstd lz4 zlibx zlib none

rsync comes with ABSOLUTELY NO WARRANTY.  This is free software, and you
are welcome to redistribute it under certain conditions.  See the GNU
General Public Licence for details.
```

3. RSYNC基本语法

格式:

```
rsync [选项] 源路径... 目标路径
```

常用选项:

- `-v`：详细模式输出，显示传输过程中的文件信息。
- `-a`：归档模式，表示递归传输文件，并保持所有文件属性。
- `-z`：对备份的文件在传输时进行压缩处理。
- `-P`：显示进度条。
- `--delete`：删除目标目录中源目录中没有的文件（可选，用于实现双向同步）
- -e：用于指定用于传输文件的远程 shell 程序 默认为 ssh，也可以选择其他如rsh或者自定义的ssh端口

常见用户示例:

```
本地同步:
rsync -av /src/directory/ /dest/directory/

远程同步:
rsync -avz /src/directory/ root@192.168.88.102:/dest/directory/

```

```
拉取远程数据到本地:
rsync -avz user@192.168.88.102:/src/directory/ /dest/directory/
```

案例说明:

- 1- 本地同步操作:

```
准备工作
    1- 在node1节点的 /usr/local目录下,创建一个nginx-1.8.0目录,并在此目录下构建一个html目录
    2- 在html目录下,创建index.html、favicon.ico、logo.png文件，以及assets目录
    3- 在assets目录下，创建index.js、index.css文件
    4- 给各个文件随机写入一些内容

cd /usr/local
mkdir -p ./nginx-1.8.0/html

cd /usr/local/nginx-1.8.0/html
touch ./index.html ./favicon.ico ./logo.png
mkdir ./assets

cd ./assets
touch index.js index.css

cd /usr/local/nginx-1.8.0/html
echo "11111 index.html" >> ./index.html
echo "22222 favicon.ico" >> ./favicon.ico
echo "33333 logo.png" >> ./logo.png

cd /usr/local/nginx-1.8.0/html/assets
echo "44444 index.js" >> ./index.js
echo "55555 index.css" >> ./index.css

# 校验:
tree nginx-1.8.0/

nginx-1.8.0/
└── html
    ├── assets
    │   ├── index.css
    │   └── index.js
    ├── favicon.ico
    ├── index.html
    └── logo.png

注意:  如果提示 tree命令不存在,  需要进行下载  (comment not find 命令找不到)
```

```
执行：dnf -y install tree

需求：对html目录进行同步操作。 将其同步到/usr/local/nginx-1.8.0/backup/下，并且如果源目录下已经删除的，在目标目录下也应该被删除

分析：
    数据源是什么： /usr/local/nginx-1.8.0/html

    目的地是什么：/usr/local/nginx-1.8.0/backup/

发现， 目标路径是不存在的， 所以先创建目标路径
mkdir -p /usr/local/nginx-1.8.0/backup/

rsync -av --delete /usr/local/nginx-1.8.0/html /usr/local/nginx-1.8.0/backup/


# 校验:
cd /usr/local
tree backup/
结果:
backup/
    ├── assets
    │   ├── index.css
    │   └── index.js
    ├── favicon.ico
    ├── index.html
    └── logo.png

说明： 部分在演示的时候， 发现没有html目录 直接就是html目录里面的内容
可能看到的是这个结果
backup/
├── assets
│   ├── index.css
│   └── index.js
├── favicon.ico
├── index.html
└── logo.png


什么原因导致的呢？ 其实是因为在写数据源路径的时候， 目录最后面带了一个 / 导致， 一旦携带了斜杠， 相当于同步是该目录下的所有内容， 并不是该目录
```

- 远程同步操作:

```
需求：对/usr/local/nginx-1.8.0/html目录进行操作，将其同步到node2的家目录的backup的目
```

```
录中

分析：
    数据源是什么：本地   /usr/local/nginx-1.8.0/html

    目的地是什么：远端  root@192.168.88.102:~/backup/

目标路径是否存在呢？ 需要创建
# 方式一： 直接在node2执行
mkdir -p ~/backup
ll ~

# 方式二： 在node1通过远程执行命令的方式
ssh root@192.168.88.102 "mkdir -p ~/backup; ls -l ~"

编写同步代码：

rsync -avz /usr/local/nginx-1.8.0/html root@192.168.88.102:~/backup/


注意： 需要双方均安装rsync，同时配置了SSH密钥（否则需要输入密码）



校验：node1执行
ssh root@192.168.88.102 "tree ~/backup"
结果为：
/root/backup
└── html
    ├── assets
    │   ├── index.css
    │   └── index.js
    ├── favicon.ico
    ├── index.html
    └── logo.png
```

4. 【综合案例】实现定时与增量备份

需求说明：

在centos stream 9的 /var/log目录下， 存储了大量的关于系统的日志：如messages文件存储大量的系统日志信息，secure日志文件存储了大量的与系统安全的日志。请对此目录中的messages、secure日志文件进行定时的备份到node2节点的/export/data/node1_backup_system_log目录中，要求每日凌晨2点进行备份一次。

```
```

```
数据同步
    数据源: node1本地
        /var/log/messages
        /var/log/secure
    目的地: 远端 node2
        root@192.168.88.102:/export/data/node1_backup_system_log

确认目的地路径是否存在: 没有就创建
    在node1远程执行命令
    ssh root@192.168.88.102 "mkdir -p /export/data/node1_backup_system_log;tree /export"

同步命令:
rsync -avz /var/log/messages /var/log/secure root@192.168.88.102:/export/data/node1_backup_system_log


定时操作:
技术点:  crontab

回顾:  crontab 基本使用格式
     格式:  crontab [选项]
         选项:
             -l  查看当前有哪些定时的任务
             -e  打开定时任务编辑窗口,  设置定时任务
     如何编写定时任务:
         分 时 日 月 周 执行的命令(命令建议是绝对路径)

     取值范围:
         分:  0~59
         时:  0~23
         日:  1~31
         月:  1~12
         周:  0~7  其中0 和 7 表示周日
     相关符号:
         *:  任意
         /:  每搁多少时间
         -:   一段连接的周期
         ,:   表示某个时间内的多个值

     如何获取命令的绝对路径:  which 命令


编写定时任务: 要求每日凌晨2点进行备份一次
0 2 * * * /usr/bin/rsync -avz /var/log/messages /var/log/secure root@192.168.88.102:/export/data/node1_backup_system_log

```

执行定时操作：

```
在node1执行：
crontab -e

输入 i 进入插入模式， 填写以下内容
0 2 * * * /usr/bin/rsync -avz /var/log/messages /var/log/secure
root@192.168.88.102:/export/data/node1_backup_system_log


输入完成后， 按下 ecs键， 输入:x 保存退出
```

5. 【综合案例】结合INOTIFY实现实时同步

需求说明： 目前在企业服务node1节点中，/export/data/logs目录是公司各个系统日志存储目录，公司希望能够实时监控备份这些日志数据到node2的/export/data/node1_exe_backup_log目录下，请运维工程师来实现。

```
分析数据同步：
    数据源：node1节点上 /export/data/logs目录中内容

    目的地：node2节点上 /export/data/node1_exe_backup_log


确认： 数据源目录 和 目的地目录是否是存在的， 如果不存在， 需要提前创建好
创建命令：
    node1执行： mkdir -p /export/data/logs
    node2执行： mkdir -p /export/data/node1_exe_backup_log


手动同步，命令如何编写呢?
node1执行： rsync -avz --delete /export/data/logs/
root@192.168.88.102:/export/data/node1_exe_backup_log


思考： 如何能够实现实时同步呢?
解决思路：
    rsync工具， 仅是一个数据同步的工具， 执行一次这个命令， 就会触发一次数据同步操作，
本身并不具备定时或者实时同步的能力，如果要做定时或者实时， 都需要借助一些其他的工具来完成
```

```
    例如： 如果要进行定时， 就可以结合linux中提供的crontab

    那么对于实时同步，最为核心的事情： 能够实时感知到目录下文件的变化(创建、修改、删除、移动)，这样只要能感知到， 我们就可以让数据同步工具触发执行一次，从而完成实时同步

在Linux中， 如果想要感知到linux目录或者文件的变化，就得使用Linux提供的INOTIFY工具
```

5.1 什么是INOTIFY

INOTIFY 是 Linux 内核的一个子系统，用于监控文件系统事件。它允许应用程序实时监控文件或目录的变化，如创建、删除、修改等。INOTIFY 通过内核通知机制，避免了轮询文件系统的高开销，提升了效率。

在 Shell 中，可以使用 `inotify-tools` 工具包来监控文件系统事件。`inotify-tools` 提供了两个主要的命令行工具：`inotifywait` 和 `inotifywatch`，其中 `inotifywait` 是最常用的工具，用于实时监控文件或目录的变化。

基本使用：

- 1- 安装INOTIFY【node1安装】

```
dnf install inotify-tools
```

```
[root@node1 ~]# dnf install inotify-tools
Last metadata expiration check: 2:06:35 ago on Sun 10 Nov 2024 11:56:21 AM CST.
No match for argument: inotify-tools
Error: Unable to find a match: inotify-tools
```

说明：在 CentOS Stream 9 中，`inotify-tools` 可能没有直接提供在默认的包仓库中。可以尝试通过安装EPEL仓库来实现安装

> EPEL（Extra Packages for Enterprise Linux）是由 Fedora 社区维护的一个软件仓库，提供了针对 RHEL（Red Hat Enterprise Linux）、CentOS 和其他 RHEL 衍生版的额外软件包，通常包含了官方仓库中没有的软件和工具

```
首先启用EPEL仓库：
dnf install epel-release

安装inotify-tools
dnf install -y inotify-tools
```

- 2- 使用方式：

- 格式： inotifywait [选项] 文件或目录

```
常用选项
    -m：持续监控（默认只监控一次后退出）。
    -r：递归监控目录及其子目录。
    -e：指定要监控的事件类型（如 create、delete、modify 等）。
    -q：静默模式，减少输出信息。
    --format：自定义输出格式。

常用事件类型
    create：文件或目录被创建。
    delete：文件或目录被删除。
    modify：文件内容被修改。
    attrib：文件属性（如权限）被修改。
    move：文件或目录被移动。
```

实例用法：

```
1. 监控目录中的文件创建和删除事件
inotifywait -m -e create,delete /path/to/dir

2. 递归监控目录及其子目录的文件修改事件
inotifywait -m -r -e modify /path/to/dir
```

重点： 结合shell脚本处理事件【模版】

```
#!/bin/bash
# 第一行为脚本文件的固定开头， 必须要携带的
# 要监控的目录
DIR="/path/to/dir"

inotifywait -m -e create,delete,modify,attrib "$DIR" | while read -r path action file; do
    if [[ "$action" == *"CREATE"* ]]; then
        echo "File created: $file"
        # 执行创建文件后的操作
    elif [[ "$action" == *"DELETE"* ]]; then
        echo "File deleted: $file"
        # 执行删除文件后的操作
    elif [[ "$action" == *"MODIFY"* ]]; then
        echo "File MODIFY: $file"
        # 执行文件内容被修改后的操作
```

```
    elif [[ "$action" == *"ATTRIB"* ]]; then
        echo "File ATTRIB: $file"
        # 执行文件被修改属性后的操作
    fi
done
```

5.2 配置INOTIFY客户端实现实时同步

- 编写INOTIFY监控脚本

```
vi /root/real_time_sync.sh

添加以下内容:

#!/bin/bash
# 定义源目录和目标目录（目标目录是远程服务器的目录）
SOURCE_DIR="/export/data/logs"
TARGET_DIR="root@192.168.88.102:/export/data/node1_exe_backup_log"  # 替换为目标服务器的用户名和目录

# 日志文件
LOG_FILE="/var/log/rsync_realtime_sync.log"

# 使用inotifywait监听${WATCH_DIR}目录中的文件变化
inotifywait -m -r -e modify,create,delete,move "${SOURCE_DIR}" | while read path action file; do
    echo "Detected ${action} on ${file} in ${path}. Starting rsync..." >> "$LOG_FILE"
    # 执行rsync同步操作
    rsync -avz --delete -e ssh "${SOURCE_DIR}/" "${TARGET_DIR}"  >> "$LOG_FILE" 2>&1
    echo "Sync completed for ${file} in ${path}." >> "$LOG_FILE"
done

```

> `inotifywait -m -r -e create,modify,delete,move`：这个命令用来监听源目录的创建、修改、删除和移动事件。`-m` 参数表示持续监听，`-r` 表示递归地监听子目录。
>
> `rsync -avz --delete -e ssh`：当监测到文件变化时，使用 `rsync` 同步源目录到远程目标目录。`-e ssh` 表示使用 SSH 连接远程服务器进行同步，`--delete` 参数保证如果源目录删除了文件，目标目录也会同步删除这些文件。
>
> 日志记录：每次文件变化和同步都会记录到 `/var/log/rsync_realtime_sync.log` 中，方便查看。

- 3- 设置脚本可执行权限：

```
chmod +x /root/real_time_sync.sh
```

- 4- 测试是否可以实时同步

```
cd /root
./real_time_sync.sh

启动后， 开启一个新窗口， 尝试向node1的/export/data/logs路径放入文件， 观察node2是否能够立即同步过来

测试成功后， 可以采用   nohup ./real_time_sync.sh & 挂载在后台使用

此种方式，有一个弊端， 如果服务器重启后， 需要手动开启实时同步脚本
```

- 5- 使用systemd创建服务（可选）

```
vi /etc/systemd/system/rsync_inotify.service

内容如下：
[Unit]
Description=Real-time Rsync Sync with Inotify
After=network.target

[Service]
ExecStart=/root/real_time_sync.sh
Restart=always
User=root
Group=root

[Install]
WantedBy=multi-user.target


保持退出后：
systemctl daemon-reload
systemctl start rsync_inotify
systemctl enable rsync_inotify
```

5.3 基于RSYNC服务的方式实现同步

5.3.1 node2设置Rsync为服务项

设置Rsync为服务型，指的是让Rsync已守护进程（daemo）的模式运行，在后台持续运行，等待来自客户端的连接和同步请求。

优势：

```
高效性：rsyncd 的守护进程模式可以避免每次连接都启动 SSH 进程，因此在某些情况下可能会比 SSH 方式略微提高性能。

简化配置：一旦配置好 rsyncd，无需每次都输入 SSH 密码，适合用于定时备份等自动化任务。
```

弊端：

```
安全性问题：rsyncd 是基于开放端口的方式进行文件同步，默认情况下并不加密传输的数据。这使得数据传输过程可能暴露给中间人攻击（MITM）。虽然可以配置 rsyncd 使用 --password-file 来限制访问，但仍然不如 SSH 安全。

需要开放端口：您需要在防火墙上开放 rsync 服务使用的端口（通常是 873），这可能增加潜在的安全风险。

配置复杂：需要在服务器上配置 rsyncd.conf，并管理模块、权限、用户等细节。
```

设置操作步骤：

- 1- 配置rsync.conf

```
vi /etc/rsyncd.conf

# 全局设置
uid = rsync
gid = rsync
use chroot = no
port = 873
max connections = 200
pid file = /var/run/rsyncd.pid
log file = /var/log/rsyncd.log
transfer logging = yes
ignore nonreadable = yes

```

```
14
15  [node1_logs]
16      path = /export/data/node1_exe_backup_log
17      comment = Backup Directory
18      read only = no
19      secrets file = /etc/rsyncd.passwd
20      hosts allow = 192.168.88.0/24
21      hosts deny = *
22      auth users = backup_user
```

- 各个参数详细说明

```
1   # 各个参数详细说明
2   # 全局设置
3   uid = rsync        # 运行rsync守护进程的用户
4   gid = rsync        # 运行rsync守护进程的组
5   use chroot = no    # 是否使用chroot环境，一般设置为no
6   port = 873 # 指定Rsync守护进程监听的端口号，默认是873。
7   max connections = 200  # 最大连接数
8   pid file = /var/run/rsyncd.pid  # 进程ID文件位置
9   log file = /var/log/rsyncd.log  # 日志文件位置
10  transfer logging = yes # 启用传输日志记录，记录每次文件传输的详细信息
11  ignore nonreadable = yes # 忽略无法读取的文件，不尝试同步它们
12
13  # 模块定义
14  [node1_logs]
15      path = /export/data/node1_exe_backup_log     # 同步的目录
16      comment = Backup Directory  # 模块描述
17      read only = no # 设置为false/no允许写操作
18      secrets file = /etc/rsyncd.passwd  # 密码文件位置
19      hosts allow = 192.168.88.0/24  # 允许访问的IP地址段
20      hosts deny = * # 拒绝所有其他主机的访问
21      auth users = backup_user # 认证同步的用户必须是backup_user，如果不配置可以是任意用户
22
23  请注意，将uid和gid设置为root以及auth users设置为root可能带来安全风险，因为这意味着任何通过认证的用户都将能够以root权限访问和修改文件。在生产环境中，通常建议创建专门的用户和组来运行Rsync守护进程，并限制对敏感文件的访问。
24
25  注意:
26      全局配置的用户， 是指的用于管理服务进程，并且执行相关的同步操作
27      在模块中定义的用户， 用于基于这个模块来干活的用户是谁， 如果不是这个用户， 无法读取这个模块相关配置
28
29  此处采用的rsync,建议可以先创建这两个用户:
```

```
useradd rsync
useradd backup_user

并为其设置密码：  均为123456
```

- 2- 创建密码文件
  - 在配置文件中指定的 `secrets file` 需要包含用户和密码，格式为 `username:password`。例如：

```
echo "backup_user:123456" > /etc/rsyncd.passwd

# 设置密码文件的权限为600，确保该文件只有 RSYNC 服务可以读取
chmod 600 /etc/rsyncd.passwd
```

- 3- 启动rsync服务

```
rsync --daemon
```

```
[root@node2 node1_exe_backup_log]# ps -ef | grep rsync
root       76372       1  0 15:02 ?        00:00:00 rsync --daemon
root       96883    9643  0 16:01 pts/0    00:00:00 grep --color=auto rsync
```

```
测试是否可以正常访问：  在node1执行
rsync rsync://backup_user@192.168.88.102/node1_logs

注意在node2需要开放端口：
firewall-cmd --permanent --add-port=873/tcp
firewall-cmd --reload
firewall-cmd --list-all
```

在配置第四项内容的时候，设置开机自启动, **请现将rsync服务关闭** 才可以进行后续的操作

```
-- 查看进程
ps -ef | grep rsync  -- 查看进程ID
-- 杀死进程
kill -9 rsync的进程ID
```

```
[root@node2 node1_exe_backup_log]# ps -ef | grep rsync
root       76372       1  0 15:02 ?        00:00:00 rsync --daemon
root       96883    9643  0 16:01 pts/0    00:00:00 grep --color=auto rsync
[root@node2 node1_exe_backup_log]# kill -9 76372
[root@node2 node1_exe_backup_log]# ps -ef | grep rsync
root       96981    9643  0 16:02 pts/0    00:00:00 grep --color=auto rsync
[root@node2 node1_exe_backup_log]#
```

- 4- 设置开机自启动

  说明： 由于rsync并不是一个系统服务（简单判断：通过systemctl启动和关闭的服务），故需要先设置为系统服务后才可以设置

```
vi /etc/systemd/system/rsyncd.service
# 添加以下内容
[Unit]
Description=Rsync Daemon
After=network.target

[Service]
ExecStart=/usr/bin/rsync --daemon --config=/etc/rsyncd.conf --no-detach
ExecReload=/bin/kill -HUP $MAINPID
Restart=always

[Install]
WantedBy=multi-user.target


特殊说明:
ExecReload=/bin/kill -HUP $MAINPID: 通过发送 SIGHUP 信号，告诉 rsync 守护进程重新加载其配置文件，而无需停止并重新启动进程。
```

- 5- 重新加载system配置并启用rsync

```
systemctl daemon-reload
systemctl enable rsyncd
systemctl start rsyncd
```

- 6- 验证服务状态

```
systemctl status rsyncd
```

```
[root@node1 ~]# systemctl status rsyncd
● rsyncd.service - Rsync Daemon
     Loaded: loaded (/etc/systemd/system/rsyncd.service; enabled; preset: disabled)
     Active: active (running) since Sun 2024-11-10 13:33:33 CST; 20s ago
    Process: 421298 ExecStart=/usr/bin/rsync --daemon --config=/etc/rsyncd.conf (code=exited, status=0/SUCCESS)
   Main PID: 421299 (rsync)
      Tasks: 1 (limit: 22930)
     Memory: 728.0K
        CPU: 2ms
     CGroup: /system.slice/rsyncd.service
             └─421299 /usr/bin/rsync --daemon --config=/etc/rsyncd.conf

Nov 10 13:33:33 node1.itcast.cn systemd[1]: Starting Rsync Daemon...
Nov 10 13:33:33 node1.itcast.cn systemd[1]: Started Rsync Daemon.
```

5.3.2 如何基于RSYNC服务进行数据同步

连接RSYNC服务的格式：

```
用户名@主机地址::模块名称  或  rsync://用户名@主机地址/模块名称

说明：
    模块名： 指的是配置到rsync.conf 中 【】的内容
    用户名： 指的是模块中指定的用户名， 如果没有指定， 可以是主机存在的用户名即可（前提是该用户需要有权限）

例如：
    backup_user@192.168.88.102::node1_logs
```

- 基于手动同步方式完成数据同步

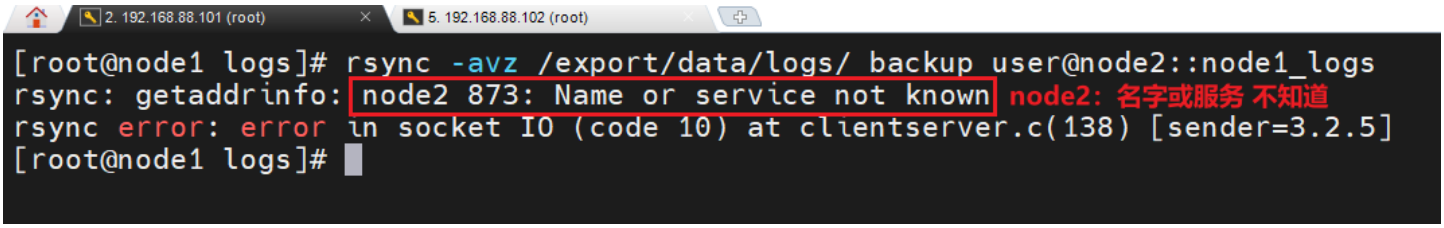
```
# 注意：在执行手动数据同步之前， 需要先在/export/data/logs/创建了日志文件， 以保证在同步的时候，有数据
echo '11111' > /export/data/logs/a.log

rsync -avz /export/data/logs/ backup_user@node2::node1_logs
```

```
[root@node1 logs]# rsync -avz  /export/data/logs/ backup_user@node2::node1_logs
Password:
sending incremental file list
./
a.log

sent 140 bytes  received 38 bytes  32.36 bytes/sec
```

可能会出现的错误：



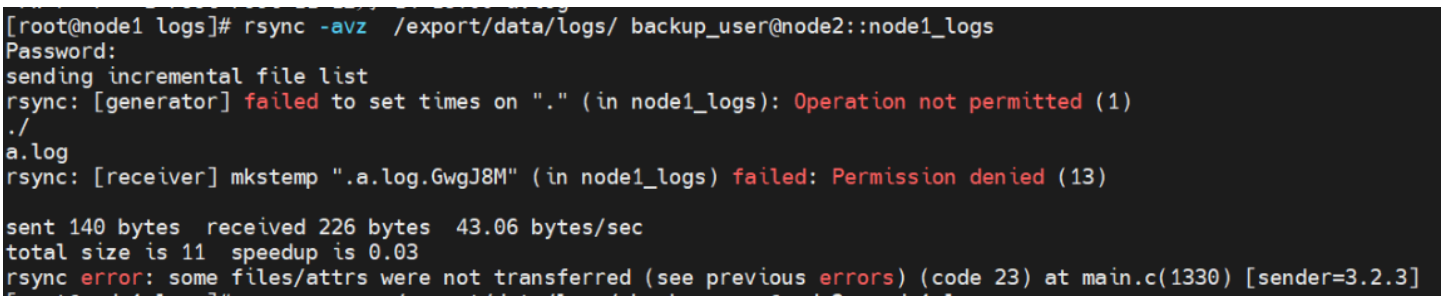

```
原因： 系统不识别node2， 并不知道node2对应服务器是谁


如何解决：
    方式一： 将更换为 IP地址访问就可以了

    方案二： 需要配置node2的对应ip的映射关系（给IP起一个别名）
        在node1中配置：
        vi /etc/hosts

        添加以下二行内容：
        192.168.88.101 node1 node1.itcast.cn
        192.168.88.102 node2 node2.itcast.cn

        建议：既然在node1都配置了， 那么node2也配置一下子， 这样就可以都是用别名来操作
```




```
原因： 由于我们在部署RSYNC服务的时候，在配置文件中，采用的是非ROOT用户， 使用rsync用户， 导致我们在进行数据同步的时候， 底层会采用rsync用户来完成数据写入， 但是该用户在写入到指定目录的时候， 并没有相对应的权限， 从而导致权限不足

解决方案： 在node2中执行
chown -R rsync:rsync /export/data/node1_exe_backup_log
```

问题2：在执行操作的时候， 每次都需要设置密码，如何解决

```
可以通过在rsync中配置 --password-file 参数， 指定一个密码文件来解决
```

具体操作：

- 步骤一： 配置密码文件

```
echo '123456' > /etc/rsync.password

chmod 600 /etc/rsync.password

注意： 权限必须是600，否则会报错
```

- 步骤二： 使用密码文件， 实现不需要输入密码

```
rsync -avz --password-file=/etc/rsync.password  /export/data/logs/
backup_user@node2::node1_logs
```

```
[root@node1 logs]# rsync -avz --password-file=/etc/rsync.password  /export/data/logs/ backup_user@node2::node1_logs
sending incremental file list

sent 75 bytes  received 12 bytes  174.00 bytes/sec
total size is 11  speedup is 0.13
```

5.3.3 实时同步脚本优化

- 编写INOTIFY监控脚本

```
vi /root/real_time_sync.sh

添加以下内容：是基于RSYNC服务的脚本文件：
#!/bin/bash
# 定义源目录和目标目录（目标目录是远程服务器的目录）
SOURCE_DIR="/export/data/logs"
#TARGET_DIR="root@192.168.88.102:/export/data/node1_exe_backup_log"  # 替换为目标服务器的用户名和目录
# 使用RSYNC服务进行数据同步
TARGET_DIR="backup_user@192.168.88.102::node1_logs"


# 日志文件
LOG_FILE="/var/log/rsync_realtime_sync.log"

# 使用inotifywait监听${WATCH_DIR}目录中的文件变化
inotifywait -m -r -e modify,create,delete,move "${SOURCE_DIR}" | while read path action file; do
```

```
    echo "Detected ${action} on ${file} in ${path}. Starting rsync..." >> "$LOG_FILE"
    # 执行rsync同步操作
    rsync -avz --port=873 --delete --password-file=/etc/rsync.password "${SOURCE_DIR}/" "${TARGET_DIR}"  >> "$LOG_FILE" 2>&1
    echo "Sync completed for ${file} in ${path}." >> "$LOG_FILE"
done
```

6. 清空今日操作所有内容

还原今日的所有操作， 以便于晚上复盘重新干：

```
node1清空操作：

# 清空自定义系统服务
systemctl stop rsync_inotify
systemctl disable rsync_inotify
rm -rf /etc/systemd/system/rsync_inotify.service
systemctl daemon-reload

# 删除脚本文件
rm -rf /root/real_time_sync.sh

# 删除 今日操作目录
rm -rf /export/data/*
rm -rf /usr/local/nginx*

# 删除rsync软件
dnf remove -y rsync
dnf clean all
rm -rf /etc/rsync.password
rm -rf /etc/rsync.conf

# 删除定时
crontab -e
进入后， 直接dd  然后输入:x 保存退出


node2执行清空：
# 清空自定义系统服务
systemctl stop rsyncd
systemctl disable rsyncd
rm -rf /etc/systemd/system/rsyncd.service
systemctl daemon-reload

```

```
# 清空相关目录
rm -rf backup a.txt
rm -rf /export/data/*

# 卸载rsync服务
dnf remove -y rsync
dnf clean all
rm -rf /etc/rsync.password
rm -rf /etc/rsync.conf

# 清空用户
userdel -rf rsync
userdel -rf backup_user
```